

1. ARCHITECTURE — HEXAGONAL (PORTS & ADAPTERS)

PATRON Mobile web	ADMIN Web dashboard	DIGITAL TWIN Simulator	MENU CACHE 10s TTL
▼ HTTP ▼			
FastAPI Gateway Rate limit · Admin token check · Route dispatch · CORS			
▼ calls core/services ▼			
Stock service	Order service	Offer service	Menu service
reserve() atomic	place_order()	evaluate_cart()	list_available()
oversell gate	update_status()	resolve_conflicts()	cache-backed
▼ IStockRepo · IOrderRepo · IOfferRepo · IMenuRepo (Ports) ▼			
SQLite Adapters stock · orders · order_items · offers · offer_redemptions · SSE event bus			

2. SERVICE BOUNDARIES & RATIONALE

Why not one monolith? Stock needs serialisable writes (no oversell ever). Menu can be stale 10s. Offers need isolated rule evaluation. One CRUD blob means stock's strict consistency infects everything else. Separate services = each tuned to its actual requirement.

Service	Consistency	Write pattern	Rationale for split
Stock	Serialisable	Atomic, high-contention	Oversell is a hard bug. Atomic SQL UPDATE WHERE qty >= ? is the only gate.
Menu	Eventual (10s ok)	Read-heavy, cached	Cache aggressively. Stock levels never cached — stale stock = oversell risk.
Orders	Eventual (lag ok)	Burst at showtimes	Burst writes at intermission. Cancel must restore stock via compensating action.
Offers	Cap atomic, reads eventual	Low write, complex read	Conflict resolution must be deterministic and testable in isolation.

3. DATA MODEL

Table	Key columns	Critical constraint
menu_items	id · name · price_paise · category · is_active	price_paise INTEGER — never floats
stock	item_id FK · quantity · last_updated	CHECK (quantity >= 0) — DB backstop
orders	id UUID · session_id · seat_number · status · discount_paise	CHECK status IN (placed preparing ready delivered cancelled)
order_items	order_id FK · item_id FK · item_name · quantity · unit_price_paise	Snapshot price at order time — menu changes don't alter history
offers	code UNIQUE · discount_type · discount_value · per_user_cap · stackable · valid_from · valid_until	Flat or percent; stackability flag; time window
offer_redemptions	offer_id FK · session_id · order_id FK · applied_at	Cap check + insert in one DB transaction

4. 25,000 CONCURRENT USERS — WALKTHROUGH

Traffic is bursty, not uniform. Spikes at showtime start (menu browse), intermission (dense ordering), credits (status polls). Digital twin models intermission as Poisson arrivals with 10× multiplier.

Endpoint	Type	Strategy	Bottleneck & mitigation
GET /api/menu	Read	In-memory cache, 10s TTL. Serve stale on DB error.	Low — cache absorbs the entire browse spike
POST /api/checkout	Write	Atomic SQL decrement. Rollback all lines on any failure.	HIGH — stock row contention. WHERE qty >= ? serialises it.
GET /api/orders/{id}	Read	No cache — status changes. 5s patron poll.	Medium — poll surge at credits; acceptable
GET /api/stock/stream	SSE	Async generator per connection. v1: polling fallback.	Medium — connection count. SSE drops to polling on disconnect.

Oversell prevention (the hard part): Two patrons both see qty=1. Both POST /checkout. SQLite serialises writers — only one UPDATE WHERE qty >= 1 matches a row. The other gets rowcount=0 → InsufficientStockError → 409 rejection. Zero oversells. Proven by concurrent thread test: 50 threads, 1 unit → exactly 1 success.

Graceful degradation: DB write failure → menu still served from cache, checkouts return 503. SSE down → patron silently falls back to polling. Offer service error → checkout proceeds without discount rather than failing the whole order.

5. ASSUMPTIONS · CLARIFICATIONS · OUT OF SCOPE

Assumptions made	Questions I would ask a PM first
• Stock is per-cinema-location, not per-screen • No patron auth — session token bound to seat + device • Admin auth via single env-var token (RBAC out of scope) • Payment fully mocked — always succeeds • Stock-sync SLA assumed < 10s (polling) / < 1s (SSE) • 25k is chain-wide; single multiplex peaks ~2,000 • All money as integer paise — never floats	• Stock per-cinema or per-screen? • One order per seat, or multiple per show? • Is seat validated against a seating map? • Are combos one SKU or multiple atomic decrements? • Real payment gateway needed or mock acceptable? • Does cancel trigger a refund flow, or just stock restore? • Exact stock-sync lag SLA — 5s? 10s? 30s? • Single-role admin or manager vs. kitchen staff?